Week 2 Module:

Date: 17/06/2026

1. Find Student Roll Number (Linear Search)

Problem Statement

Your class teacher has a list of student roll numbers stored in the order they registered. Since the list is unsorted, you need to find whether a particular student roll number exists.

Write a program to search for the given roll number using **Linear Search**.

Input Format

- First line contains integer N (number of students)
- Second line contains N roll numbers
- Third line contains integer X (roll number to search)

Output Format

- Print "Found at index i" if present
- Otherwise print "Not Found"

Constraints

- $1 \leq N \leq 1000$

Example

Input

5
101 105 113 110 130
110

Output

Found at index 3

(1) find student Roll Numbers (linear search)

```
#include <stdio.h>
int main() {
int N, i, found = 0;
scanf ("%d", &N);
int roll_numbers[N];
for (int i = 0; i < N; i++) {
    scanf ("%d", &roll_numbers[i]);
}
scanf ("%d", &X);
for (int i = 0; i < N; i++){
    if (roll_numbers[i] == X) {
        printf ("found at index %d\n", i);
        found = 1;
        break;
    }
}
if (!found){
    printf ("Not found \n");
}
return 0;
}
```

Input :—

```
4
101  343  400  432  343
454
```

O/P :— found at index 1

main.c

Share Run

```c
#include <stdio.h>

int main() {
    int N, X, found = 0;

    scanf("%d", &N);

    int roll_numbers[N];

    // Read roll numbers
    for (int i = 0; i < N; i++) {
        scanf("%d", &roll_numbers[i]);
    }

    scanf("%d", &X);

    // Search for the roll number
    for (int i = 0; i < N; i++) {
        if (roll_numbers[i] == X) {
            printf("Found at index %d\n", i);
            found = 1;
```

main.c

Share Run

```c
    scanf("%d", &roll_numbers[i]);
}

scanf("%d", &X);

// Search for the roll number
for (int i = 0; i < N; i++) {
    if (roll_numbers[i] == X) {
        printf("Found at index %d\n", i);
        found = 1;
        break;
    }
}

// If not found
if (!found) {
    printf("Not Found\n");
}

return 0;
}
```

Output Clear

```
4
101 343 400 432 343 454
Found at index 1


=== Code Execution Successful ===
```

2. Search Product ID (Binary Search)

Problem Statement

An online store keeps product IDs sorted in ascending order. A customer wants to know whether a particular product exists in inventory.

Write a program to search the product using **Binary Search.**

Input Format

- First line contains N
- Second line contains sorted product IDs
- Third line contains target ID

Output Format

Print:

Product Available

or

Product Not Available

Constraints

- 1 ≤ N ≤ 10^5

Example

Input

6
10 20 30 40 50 60
40

Output

Product Available

(2) Search product ID (binary search)

```
#include <stdio.h>
int main () {
int N, target, found = 0;
Scanf ("%d", & N);
int product_id[N];
for (int i = 0; i < N; i++) {
    Scanf ("%d", & product_id[i]);
}
Scanf ("%d", & target);
int low = 0;
int high = N - 1;
int mid = (low + high - low) / 2;
if (product_id[mid] == target) {
    printf ("product Available \n");
    found = 1;
    break;
} else if (product_id[mid] < target) {
    low = mid + 1;
} else {
    high = mid - 1;
}
if (! found) {
    printf ("product Not Available \n");
}
return 0;
}
```

I/P:- 6

10 20 30 40 50 60

40

O/P:-

Product Available.

main.c

Share Run

```c
#include <stdio.h>

int main() {
    int N, target, found = 0;

    scanf("%d", &N);

    int product_ids[N];

    // Read product IDs
    for (int i = 0; i < N; i++) {
        scanf("%d", &product_ids[i]);
    }

    scanf("%d", &target);

    int low = 0;
    int high = N - 1;

    // Binary search
    while (low <= high) {
```

main.c

Share Run

```
20      // Binary search
21      while (low <= high) {
22      |   int mid = low + (high - low) / 2;
23      
24      |   if (product_ids[mid] == target) {
25      |   |   printf("Product Available\n");
26      |   |   found = 1;
27      |   |   break;
28      |   }
29      |   else if (product_ids[mid] < target) {
30      |   |   low = mid + 1;
31      |   }
32      |   else {
33      |   |   high = mid - 1;
34      |   }
35      }
36      
37      // If product is not found
38      if (!found) {
39      |   printf("Product Not Available\n");
40      }
```

main.c

Share Run

```
23
24      if (product_ids[mid] == target) {
25          printf("Product Available\n");
26          found = 1;
27          break;
28      }
29      else if (product_ids[mid] < target) {
30          low = mid + 1;
31      }
32      else {
33          high = mid - 1;
34      }
35  }
36
37  // If product is not found
38  if (!found) {
39      printf("Product Not Available\n");
40  }
41
42  return 0;
43  }
```

Output

Clear

```
4
10 20 30 40
20
Product Available


=== Code Execution Successful ===
```

3. Arrange Exam Scores (Bubble Sort)

Problem Statement

A teacher wants to arrange students' exam scores in ascending order to prepare the result sheet.

Write a program using **Bubble Sort**.

Input Format

- First line contains N
- Second line contains N integers

Output Format

Print the sorted array.

Example

Input

5
89 45 78 12 99

Output

12 45 78 89 99

③ Arrange Exam Scores (bubble Sort)

```
#include <stdio.h>
int main () {
int N;
scanf ("%d", &N);
int scores[N];
for (int i = 0; i < N; i++) {
   scanf ("%d", &scores[i]);
}
for (int i = 0; i < N-1; i++) {
 for (int j = 0; j < N-i-1; j++) {
   if (scores[j] > scores[j+1]) {
     int temp = scores[j];
     scores[j] = scores[j+1];
     scores[j+1] = temp;
  } } }
for (int i = 0; i < N; i++) {
  printf ("%d", scores[i]);
  if (i < N-1) {
    printf (" ");
  } }
printf ("\n");
return 0;
}
```

I/p :- 5
89 45 91 12 99

O/p :- 12 45 78 89 99

main.c

Share Run

```c
            if (scores[j] > scores[j + 1]) {
                int temp = scores[j];
                scores[j] = scores[j + 1];
                scores[j + 1] = temp;
            }
        }
    }

    // Print sorted scores
    for (int i = 0; i < N; i++) {
        printf("%d", scores[i]);

        if (i < N - 1) {
            printf(" ");
        }
    }

    printf("\n");

    return 0;
}
```

main.c

Share Run

```c
#include <stdio.h>

int main() {
    int N;
    scanf("%d", &N);

    int scores[N];

    // Read scores
    for (int i = 0; i < N; i++) {
        scanf("%d", &scores[i]);
    }

    // Bubble Sort
    for (int i = 0; i < N - 1; i++) {
        for (int j = 0; j < N - i - 1; j++) {
            if (scores[j] > scores[j + 1]) {
                int temp = scores[j];
                scores[j] = scores[j + 1];
                scores[j + 1] = temp;
            }
```

Output Clear

```
6
20 10 3 40 50 30
3 10 20 30 40 50


=== Code Execution Successful ===
```

4. Arrange Library Book Numbers (Insertion Sort)

Problem Statement

A librarian receives book serial numbers one by one and wants to place each new book in the correct position.

Write a program using **Insertion Sort** to arrange book numbers.

Input Format

- First line contains N
- Second line contains N integers

Output Format

Print sorted book numbers.

Example

Input

5
55 23 87 11 34

Output

11 23 34 55 87

(4) Arrange Library Book Numbers (insertion sort)

```
#include <stdio.h>
int main () {
int N;
Scanf ("%d", &N);
int books [N];
for (int i = 0; i < N; i++) {
    scanf ("%d", & books [i]);
}
for (int i = 1; i < N; i++) {
  int key = books [i];
  int j = i - 1;
  while (j >= 0 && books [j] > key) {
    books [j + 1] = books [j];
    j--;
  }
  books [j + 1] = key;
}
for (int i = 0; i < N; i++) {
  printf ("%d", books [i]);
  if (i < N - 1) {
    printf (" ");
  }
}
printf ("\n");
return 0;
}
```

I/p:— 6

20 10 3 40 50 30

O/p:—

3 10 20 30 40 50

main.c

Share Run

```c
#include <stdio.h>

int main() {
    int N;
    scanf("%d", &N);

    int books[N];

    // Read book numbers
    for (int i = 0; i < N; i++) {
        scanf("%d", &books[i]);
    }

    // Insertion Sort
    for (int i = 1; i < N; i++) {
        int key = books[i];
        int j = i - 1;

        while (j >= 0 && books[j] > key) {
            books[j + 1] = books[j];
            j--;
```

main.c

Share Run

```c
    while (j >= 0 && books[j] > key) {
        books[j + 1] = books[j];
        j--;
    }

    books[j + 1] = key;
}

// Print sorted books
for (int i = 0; i < N; i++) {
    printf("%d", books[i]);

    if (i < N - 1) {
        printf(" ");
    }
}

printf("\n");

return 0;
}
```

Output

Clear

```
5
23 45 10 2 33
2 10 23 33 45


=== Code Execution Successful ===
```

5. Rank Players by Score (Selection Sort)

Problem Statement

A gaming club wants to rank players based on scores from lowest to highest.

Write a program using **Selection Sort**.

Input Format

- First line contains N
- Second line contains player scores

Output Format

Print sorted scores.

Example

Input

5
70 20 90 10 40

Output

10 20 40 70 90

(5) Rank Players by score (selection sort)

```
#include <stdio.h>
int main () {
int N;
scanf ("%d", &N);
int scores[N];
for (int i = 0; i < N; i++) {
    scanf ("%d", &scores[i]);
}
for (int i = 0; i < N-1; i++) {
 int min_idx = i;
 for (int j = i+1; j < N; i++) {
    if (scores[j] < scores[min_idx]) {
      min_idx = j;
    }
 }
 int temp = scores[min_idx];
 scores[min_idx] = scores[i];
 scores[i] = temp;
}
for (int i = 0; i < N; i++) {
  printf ("%d", scores[i]);
  if (i < N-1) {
    printf (" ");
  }
printf ("\n");
return 0;
}
```

I/p:-

```
5
90 20 90 10 40
```

O/p:-

```
10 20 40 70 90
```

main.c

Share Run

```c
#include <stdio.h>

int main() {
    int N;
    scanf("%d", &N);

    int scores[N];

    // Read scores
    for (int i = 0; i < N; i++) {
        scanf("%d", &scores[i]);
    }

    // Selection Sort
    for (int i = 0; i < N - 1; i++) {
        int min_idx = i;

        for (int j = i + 1; j < N; j++) {
            if (scores[j] < scores[min_idx]) {
                min_idx = j;
            }
```

main.c

Share Run

```c
            }
        }

        int temp = scores[min_idx];
        scores[min_idx] = scores[i];
        scores[i] = temp;
    }

    // Print sorted scores
    for (int i = 0; i < N; i++) {
        printf("%d", scores[i]);

        if (i < N - 1) {
            printf(" ");
        }
    }

    printf("\n");

    return 0;
}
```

Output

Clear

```
6
34 53 13 445 22 45
13 22 34 45 53 445


=== Code Execution Successful ===
```